

Spring AI: Modül 12

Canlı Ortama Dağıtım & GraalVM Native AOT

Saniyenin Altında Başlatma ve Yüksek Verimli Bulut Yapay Zekası

1. Bulut Odaklı Yapay Zeka Dağıtım Zorlukları

- Geleneksel JVM iş yükleri bulut ve sunucusuz (serverless) ortamlarda şu engellerle karşılaşır:
 - **Yavaş Soğuk Başlatma (Cold Starts):** JIT derlemesi ve sınıf yolu taraması saniyeler sürer.
 - **Yüksek Bellek Ayak İzi:** JVM bellek maliyeti yatay ölçeklendirmeyi pahalılaştırır.
 - **Oto-ölçekleme Gecikmesi:** Ani yapay zeka istek trafiğinde yeni konteynerler anında yanıt veremez.
- GraalVM Native Image, Spring Boot 3.4+ ve Spring AI

2. GraalVM Native AOT Temelleri

- **Zamanından Önce Derleme (AOT):**
 - Kapalı dünya varsayımı (Closed-world assumption): Tüm sınıflar, metotlar ve yansıma (reflection) bilgisi derleme anında bilinmelidir.
 - Kullanılmayan bayt kodları ve dinamik yükleme mekanizmaları temizlenir.
 - Anında en yüksek performansta başlayan tek parça ikili dosya üretilir.
- Sağladığı Faydalar:
 - Başlatma süresi 3-5 saniyeden **< 50 milisaniyeye** düşer.

3. Spring AI Çalışma Zamanı İpuçları (`RuntimeHintsRegistrar`)

- Spring Boot 3.4, dinamik yansımada ve dosya kaynaklarını tanıtmak için `RuntimeHintsRegistrar` sunar:
 - **Alan Modeli Yansımada:** Prompt POJO'ları, JSON şemaları, kayıtlar (record) ve yapısal çıktı sınıfları.
 - **Kaynak Dosyaları:** Prompt şablonları (`.st`), vektör ETL dokümanları ve sözlük dosyaları.

4. Çok Aşamalı Distroless Docker Mimarisi

- Üretim konteynerleri olabildiğince yalın ve güvenli tutulmalıdır:
 - Aşama 1: `native-image-maven-plugin` veya Cloud Native Buildpacks ile yerel ikili oluşturma.
 - Aşama 2: İkiliyi Google Distroless (`gcr.io/distroless/cc-debian12`) içine yerleştirme.
 - Sıfır kabuk (shell), sıfır paket yöneticisi ve minimum CVE saldırı yüzeyi.

5. Actuator Yapay Zeka Sağlık Göstergeleri & Sanal İş Parçacıkları

- **Spring Boot Actuator Entegrasyonu:**
 - Özel `AiModelHealthIndicator` bileşeni model uç noktalarının ve vektör veri tabanlarının canlılığını denetler.
 - Gecikme (latency) ve erişim durumunu `/actuator/health` üzerinden sunar.
- **Project Loom Sanal İş Parçacıkları (Virtual Threads):**
 - `spring.threads.virtual.enabled=true` ayarı, LLM akışlarında ve araç çağrılarında yüksek eşzamanlı I/O verimliliği sağlar.

6. Özet ve Canlı Dağıtım Kontrol Listesi

1. Dinamik POJO ve şablon dosyaları için her zaman `RuntimeHintsRegistrar` yazın.
2. Yerel ikiliyi canlıya almadan önce CI ortamında test edin.
3. Distroless ve Rootless konteynerlerle güvenliği sıkılaştırın.
4. Model yanıt sürelerini ve erişilebilirliğini Actuator Health ile izleyin.